

StockTracker Pro Architecture

StockTracker Pro is a vanilla JavaScript single-page app that tracks stock quotes, a watchlist, and a simple portfolio summary.

Project Structure

- `index.html` defines the layout and mounts the app sections.
- `styles.css` handles the responsive UI and visual design.
- `config.js` stores the local Alpha Vantage and Twelve Data API keys outside the main app logic.
- `app.js` contains all application behavior, data fetching, rendering, and persistence.

Architecture Overview

The app follows a simple client-side architecture:

1. The HTML file loads the stylesheet, the local config module, and the main app module.
2. `app.js` waits for `DOMContentLoaded`, then wires the event listeners and initializes state.
3. User search input is resolved against a small local company catalog first.
4. If the input looks like a company name, the app calls the Alpha Vantage `SYMBOL_SEARCH` endpoint to return matching symbols.
5. The user selects the correct symbol from the rendered matches and the app then calls the Alpha Vantage `GLOBAL_QUOTE` endpoint with `fetch`.
6. If Alpha Vantage reports a daily token limit issue, the app retries the lookup with the Twelve Data `time_series` endpoint.
7. If both live quote services fail, the app falls back to local demo data.
8. The returned quote is normalized into a shared stock object shape.
9. The UI is re-rendered from that state for the stock details, watchlist, and portfolio summary.

Key Data Flow

Search Flow

- User enters a symbol or company name.
- The app validates the query and checks whether it is a symbol or a company name.
- Company-name queries request matching symbols from Alpha Vantage `SYMBOL_SEARCH`.
- The selected symbol requests the latest quote from Alpha Vantage `GLOBAL_QUOTE`.
- If Alpha Vantage hits a daily token limit, the app retries with Twelve Data `time_series`.
- If both live quote services fail, the app falls back to cached or local demo data.
- The active quote is stored as the current stock and rendered into the details panel.

Watchlist Flow

- Stocks can be added or removed from the watchlist.
- Watchlist symbols are persisted in `localStorage`.
- On load, the app restores the watchlist and refreshes the visible quotes.

Portfolio Flow

- Users enter the number of shares they own for the active stock.
- Share counts are stored in `localStorage` by symbol.
- The portfolio summary multiplies shares by the current quote price to calculate position value and total value.
- The Portfolio Reset button clears the saved portfolio shares and reloads the page.

State and Persistence

The application keeps its runtime state in a single in-memory object inside `app.js`.

Persisted keys:

- `stocktracker.portfolio` for share counts
- `stocktracker.watchlist` for saved symbols
- `stocktracker.quote-cache` for quote snapshots used during refreshes and fallback rendering

API Integration

The app uses REST GET requests to:

`https://www.alphavantage.co/query?function=SYMBOL_SEARCH&keywords=Apple&apikey=...`

`https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol=AAPL&apikey=...`

`https://api.twelvedata.com/time_series?symbol=AAPL&interval=1min&apikey=...`

The response is normalized so the rest of the app can render a consistent structure for price, change, percentage change, open, high, low, and volume.

UI Responsibilities

- `index.html` provides sections for search, stock details, watchlist, and portfolio summary.
- `styles.css` handles responsiveness, card layout, spacing, and positive/negative value styling.
- `app.js` updates the DOM through dedicated render functions rather than scattered inline logic.

Security Note

The Alpha Vantage and Twelve Data keys are stored in `src/config.js` instead of the main source file so they can stay local and out of the primary app logic. The file is ignored by Git.

Setup Instructions

Follow these steps to run StockTracker locally:

1. Open the project folder in your editor.
2. Confirm that `src/config.js` exists and contains your API keys:

```
export default {  
  ALPHA_VANTAGE_API_KEY: "YOUR_ALPHA_VANTAGE_KEY",
```

```
TWELVE_DATA_API_KEY: "YOUR_TWELVE_DATA_KEY",  
};
```

3. Start a local web server from the project folder so the browser can load the ES modules properly:

```
cd src  
python -m http.server 8000
```

4. Open the app in your browser:

```
http://localhost:8000/index.html
```

5. Search for a stock symbol or company name to begin tracking quotes.

Alternative

If you are using VS Code, you can also open the project with Live Server or a simple local preview extension and run `src/index.html` from there.

The app is designed to load from the same folder as `config.js`, and a local HTTP server is the most reliable way to run it.